

Integrating Harness Events into PreviouslyUpcoming

Oct 4, 2026 · @Mikkel

Summary

PreviouslyUpcoming (PU) becomes the place where Harness data is collected, priced, reviewed and turned into calibration suggestions. It reads both Harness metadata outputs, Routing log v1 and Harness Events v1, defined in [Appendix A](#), joins it with the session transcripts PU already captures for its cost overview, and answers three questions: what did each task cost, did it land in the right tier, and what should change in `routing.yaml`.

PU consumes the stream; it never writes to it and never changes Harness behaviour directly. Every change to routing goes through a `routing.yaml` pull request a human approves.

Goals

- **One place to look:** Harness outcomes sit next to session memory, handover and cost in the tool already used for Claude Code work.
- **Cost per task and per tier**, not just per session, so the routing question ("does this save money?") has a direct answer.
- **Baseline versus routed comparison** that controls for task difficulty.
- **Calibration as a product feature:** evidence-backed suggestions, delivered as reviewable changes.

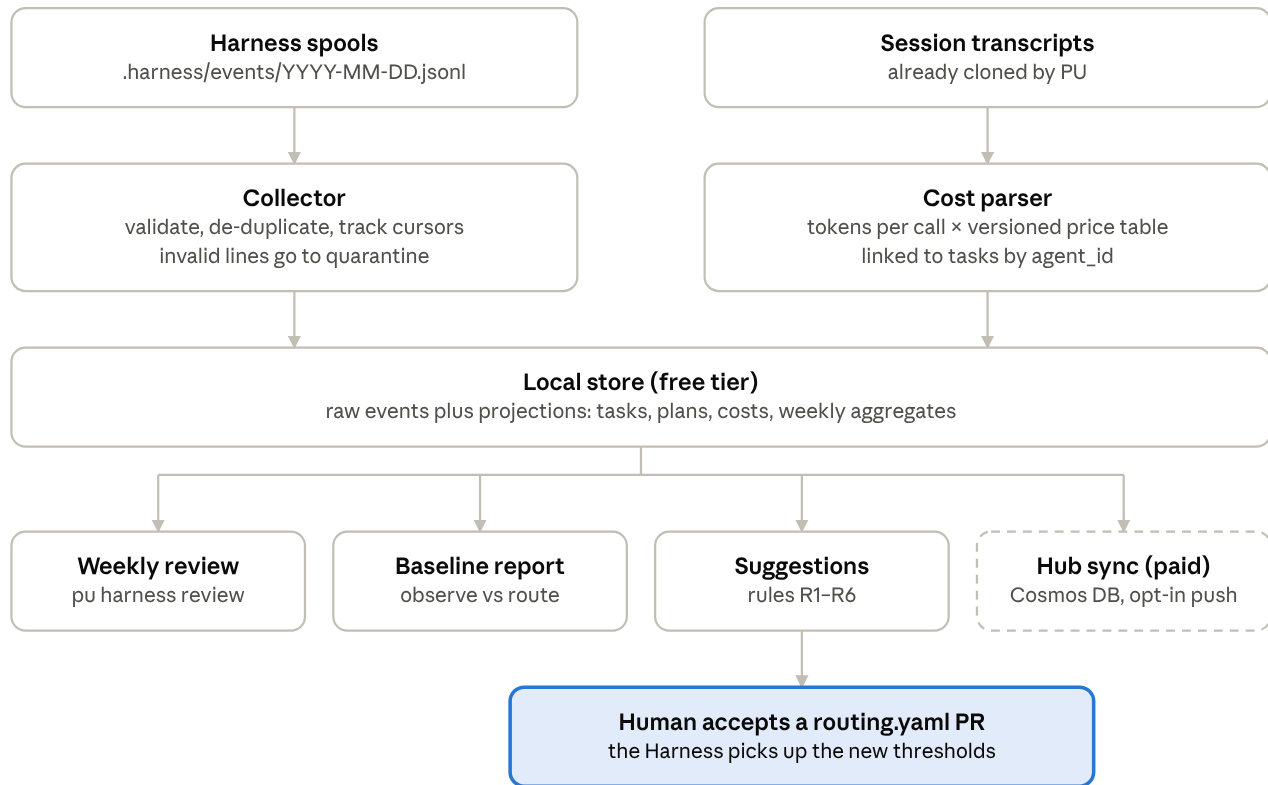
Non-goals

- Re-implementing the Harness or enforcing anything at commit time. The commit gate stays local and synchronous.
- Storing code, prompts or diffs from the event stream. Events don't carry them, and PU's transcript features are governed by their own rules.
- Individual performance tracking. Reviews are for the person running the sessions; see Product boundaries.

Architecture

Two inputs feed one local store: the Harness event spools and the session transcripts PU already clones. Everything PU shows is a projection over that store, and the only path back into the Harness is a `routing.yaml` change a human accepts.

PU reads the stream and prices it; only a human changes routing



PU integration · 2 sources, 1 store, 4 outputs, 1 human gate

The left column is new work; the right column extends PU's existing transcript and cost pipeline. The dashed hub box is the optional paid tier.

Ingestion

A PU collector tails each repository's spool, validates every line against the v1 JSON Schema, and upserts events by `event_id`. It runs on PU's existing session hooks plus a periodic sweep, so ingestion keeps up without a daemon.

Two inputs from the Harness

Output	File	De-duplication key	PU uses it for
Routing log v1	<code>.harness/routing-log/YYYY-MM.jsonl</code>	<code>task_id + highest revision</code>	The primary source for task records, tiers, eval history and the agent IDs used for cost
Harness Events v1	<code>.harness/events/YYYY-MM-DD.jsonl</code>	<code>event_id</code>	Timing, gate activity, parallel behaviour, and tasks still in progress

Both files are tailed by the same collector with the same cursor, validation and quarantine rules, following the Harness's `INGESTION.md`.

Triggers

Trigger	What it does
PU's existing <code>SessionEnd / Stop</code> hook	Ingests the spool of the repository the session ran in
<code>pu harness sync</code>	Manual sweep of all registered repositories
Daily ETL run (already planned for "what was done today")	Sweeps everything, then builds projections

Discovery

Repositories register themselves: the first time the collector sees a `harness.doctor` event or a `.harness/events/` directory in a session's working directory, it adds the repo to PU's registry with its spool path and `repo.remote_sha256`.

Cursors

Per spool file, the collector stores `{file, inode, byte_offset, last_event_id}`. It reads from the offset to the last complete newline and never reads a partial trailing line. A changed inode or a file shorter than the offset means the file was rotated or replaced; the collector re-reads it from the start and relies on de-duplication.

Validation and quarantine

- Lines that fail JSON parsing or schema validation go to a quarantine table with the raw line, file, offset and error. They never block ingestion of later lines.
- Unknown `type` values and unknown fields are accepted and stored (forward compatibility, per the v1 versioning rules), and flagged so PU can show "events from a newer Harness version".
- A `schema` other than `harness.events/v1` is routed to the matching parser if PU has one, otherwise quarantined.

Idempotency and ordering

- Upsert on `event_id`; re-ingesting a file is always safe.
- Projections sort by `ts`, then `event_id`, because parallel subagents can write slightly out of order.
- A projection that depends on a missing earlier event (for example `task.completed` with no `task.scored`) is marked incomplete rather than guessed.

Retention handshake

The Harness deletes spool files after `events.retention_days` (30 by default). PU warns when a registered repo hasn't been ingested for more than half that window, so data is never lost to rotation.

Storage model

Events land in a local store first and sync to the hub only if the user pushes. Raw events are kept as-is; everything PU shows is a projection rebuilt from them, so a bug in a projection is fixed by rebuilding, never by editing data.

Local and hub

Layer	Store	Holds	Tier
Local	Embedded database on the developer's machine (SQLite or DuckDB)	Raw events, quarantine, cursors, local projections	Free

Layer	Store	Holds	Tier
Hub	Cosmos DB, PU's planned document store	Pushed raw events and projections, across machines and repos	Paid

Collections

Collection	One record per	Partition key (hub)	Built from
<code>harness_events</code>	Event	<code>/repo_month</code> (<code><remote_sha256>#YYYY-MM</code>)	Ingestion
<code>harness_quarantine</code>	Rejected line	<code>/repo_month</code>	Ingestion
<code>harness_tasks</code>	Task	<code>/repo_key</code>	Projection over task, subagent, escalation and eval events
<code>harness_plans</code>	Plan	<code>/repo_key</code>	Projection over plan events and its tasks
<code>harness_costs</code>	Subagent run or orchestrator window	<code>/repo_key</code>	Cost attribution (next section)
<code>harness_weekly</code>	Repo × ISO week × mode	<code>/period</code>	Weekly aggregation
<code>harness_suggestions</code>	Calibration suggestion	<code>/repo_key</code>	Calibration rules

Raw routing-log records are kept in their own collection, `harness_routing_log` (partitioned like `harness_events`). The `harness_tasks` projection is seeded from the latest routing-log revision per task and enriched from events, so tasks still in progress (no routing-log line yet) appear from events alone and are marked incomplete.

Task record (projection)

```
{
  "id": "svc-booking:PLAN-42.2",
  "repo_key": "9f2c...",
  "plan_id": "PLAN-42",
  "mode": "route",
  "scores": {"a": 0, "b": 1, "c": 2, "n": 0, "r": 0, "v": 1},
  "total": 4,
  "score_band": "T2",
  "tier_planned": "T4",
  "overrides": ["security_coupling"],
  "final_tier": "T4",
  "agent_ids": ["a19c", "a7f3"],
  "eval_rounds": 2,
  "escalations": 0,
  "redispatches": 0,
  "outcome": "pass",
  "lines_changed": 212,
  "cost_usd": 0.84,
  "wall_clock_s": 1140,
  "release_failed": false,
  "complete": true
}
```

`score_band` is the tier the raw score implies before overrides; `tier_planned` is what was actually chosen. Keeping both lets PU tell rubric problems from override problems.

Cost attribution

Harness events carry no token counts by design. PU prices each subagent run from the transcripts it already clones, links runs to tasks through `agent_id`, and reports orchestration overhead separately so it is never hidden inside task costs.

Sources

Data	Where it comes from
Token usage per model call (input, output, cache write, cache read)	Claude Code transcripts PU already clones: the main session file and the subagent files under <code><session>/subagents/agent-<agent_id>.jsonl</code>
Model per call	The same transcript entries
Price per model and token type	A versioned price table in PU, with an effective-from date per row
Agent ↔ task link	<code>subagent.stopped</code> events: <code>agent_id</code> plus <code>task_ids[]</code>
Plan time window	<code>plan.created</code> to the plan's last <code>task.completed</code>

Linking a subagent run to its task

The Harness already requires every implementer, evaluator and architect report to start with a header such as `DONE: PLAN-42.2` or `PASS: PLAN-42.2, PLAN-42.3`. The `require-report` hook parses that header and emits it as `task_ids[]` on `subagent.stopped`. This is a small producer addition, made as an additive v1 change in Appendix A.

For completed tasks the link is even simpler: the routing log's `runs[]` lists every agent run for the task with its `agent_id`, so PU prices those runs and sums them. The `subagent.stopped` link is the fallback for tasks without a routing-log line yet.

Allocation rules

1. **Subagent runs** are priced call by call and assigned to their `task_ids`. A run covering several tasks (one eval for a parallel group) is split evenly across them.
2. **Orchestrator overhead** is the main-session cost inside a plan's time window. It is stored as its own `harness_costs` record per plan and shown as a separate line, "orchestration", in every report.
3. **Fully loaded task cost**, used for comparisons, adds the task's share of its plan's orchestration overhead, split in proportion to subagent cost.

4. **Repricing:** costs are computed from tokens and the price table at the call's date. A new price row never rewrites history unless the user asks for a "what if at today's prices" view.

Plans billed by subscription

On a subscription, dollars are not what is spent: usage limits are. PU shows list-price equivalents with a clear label, plus token totals per model, which map more directly to limit consumption.

Baseline and comparison

PU answers "does routing save money?" by comparing tasks from `observe` weeks with tasks from `route` weeks **within the same score band**. In observe mode the orchestrator still scores every task but doesn't route, so both periods carry the same difficulty measure, and a week of unusually hard tasks can't distort the result.

Method

- 1. **Baseline:** 1-2 weeks with `mode: observe` on the pilot repo, aiming for at least 30 completed tasks.
- 2. **Treatment:** switch to `mode: route` and collect at least the same number of tasks.
- 3. **Stratify:** group tasks by `score_band` (T1-T4 from raw score, before overrides).
- 4. **Compare per band:** median fully loaded cost per task, median cost per changed line, first-try pass rate, eval rounds, escalations, release failures within 7 days.
- 5. **Weight:** combine bands using the treatment period's task mix, so the headline reflects the work actually being done now.
- 6. **Report:** a headline delta with the per-band table behind it, and a sample-size warning for any band with fewer than 8 tasks.

Report shape

Score band	Tasks (observe / route)	Median cost per task	First-try pass	Eval rounds	Release failures
T1	n / n	\$ → \$ (Δ%)	% → %	x → x	n → n
T2	...	...	...	...	...
T3	...	...	...	...	...
T4	...	...	...	...	...

Score band	Tasks (observe / route)	Median cost per task	First-try pass	Eval rounds	Release failures
Weighted	n / n	Δ%	Δ pp	Δ	Δ

The orchestration overhead line is shown above the table, so it's visible whether routing savings exceed the cost of the orchestrator itself.

Caveats PU should state in the report

- Observe mode isn't a pure "no Harness" baseline: the commit gate and eval already run, so it measures routing, not the whole Harness. A "gate off" baseline is possible but not recommended in production repos.
- Model or price changes between periods are flagged; the comparison can be recomputed at a single price table.
- Medians, not means, because a few escalated tasks dominate averages.

Weekly review

The weekly review is a PU view and a CLI command (`pu harness review --week 2026-w42`) built from `harness_tasks` , `harness_costs` and `harness_weekly` . It replaces the review that used to be described in the Harness itself.

Metrics

Metric	Definition	Starting target	Read it as
First-try pass rate per tier	Tasks with <code>outcome = pass_first_try</code> ÷ completed tasks, per <code>final_tier</code>	≥ 70%	Below target: tasks land in a tier that can't handle them
Escalation rate per tier	Tasks with ≥ 1 escalation ÷ tasks dispatched to that tier	≤ 15%	Above target: that tier's upper threshold is too high
Override rate	Tasks where <code>tier_planned ≠ score_band</code> ÷ all tasks	Watch the trend	Rising: the rubric misses something the overrides keep catching

Metric	Definition	Starting target	Read it as
Eval rounds per task	Mean <code>eval_rounds</code>	≤ 2	High: acceptance criteria or tests are unclear
Gate denials	<code>gate.decision = deny</code> count, by <code>reason</code>	Watch the trend	Frequent "stale marker": eval is too slow or runs too early
Merge conflicts	<code>task.redispached</code> with <code>merge_conflict</code> ÷ parallel tasks	$\leq 10\%$	High: parallel groups overlap more than the plan claims
Cost per task by tier	Median fully loaded cost per <code>final_tier</code>	T2 well below T4	Flat across tiers: routing isn't saving anything
Orchestration share	Orchestration overhead ÷ total cost	Watch the trend	High: the orchestrator is too heavy for the work
Release failures	<code>release.failed</code> linked to the week's commits	Trending down	The local gate misses what the release gate catches
Human interventions	<code>human.intervention</code> count, by <code>reason</code>	Trending down	A class of task the system can't handle yet

Layout

1. **Headline:** total cost, tasks completed, first-try pass rate, change versus the previous week.
2. **Tier table:** one row per tier with the metrics above.
3. **Cost split:** implementation, evaluation, re-planning, orchestration.
4. **Exceptions:** escalated tasks, human interventions and release failures, each linked to its PU session summary and handover memory.
5. **Suggestions:** open calibration suggestions (next section).

The exception links are where PU adds value no plain dashboard has: each escalated task opens straight into the session summary that explains what happened.

Calibration suggestions

PU turns the calibration rules into evidence-backed suggestions, each one a concrete `routing.yaml` change with the data that justifies it. A human accepts or rejects; PU never edits a repository on its own.

Rules

These are the rules that used to live in the Harness, now owned by PU:

Rule	Condition (over the last 4 weeks, minimum 20 tasks in scope)	Suggested change
R1 Tier too weak	Escalation rate above 25% in a tier	Lower that tier's <code>max_score</code> by 1
R2 Tier too strong	First-try pass above 90% in T3 or T4, with median diff under the repo's median	Raise the tier below's <code>max_score</code> by 1
R3 Dimension blind spot	One rubric dimension scored 0–1 on more than half of escalated tasks, while escalation briefs name that area	Add an override rule, or reword the dimension in the rubric skill
R4 Escalation threshold	Most escalations trigger on test failures and resolve without a re-plan changing scope	Raise <code>escalation.test_failures</code> by 1
R5 Evaluator too weak	T3 tasks with <code>PASS</code> later linked to a <code>release.failed</code>	Set <code>evaluator.model_by_tier.T3: opus</code>
R6 Parallelism too high	Merge-conflict rate above 10%	Lower <code>parallel.max</code> by 1

Only one open suggestion per `routing.yaml` key at a time, and a key changed in the last 7 days is not suggested again, so effects can be measured one change at a time.

Suggestion record

```
{
  "id": "sug-2026-W44-svc-booking-R1",
  "repo_key": "9f2c...",
  "rule": "R1",
  "status": "open",
  "summary": "T2 escalation rate is 31% (9 of 29 tasks); lower T2 max_score
from 6 to 5",
  "evidence": { "window": "2026-W41..W44", "tasks": 29, "escalated": 9,
"task_ids": ["PLAN-51.3", "PLAN-55.1"] },
  "patch": { "path": "tiers.T2.max_score", "from": 6, "to": 5 },
  "created": "2026-11-02"
}
```

Statuses: `open`, `accepted`, `rejected`, `superseded`, `applied`. A suggestion moves to `applied` when PU sees events from the repo carrying a `routing.yaml` version that contains the change.

Delivery

- **In PU:** suggestions appear at the end of the weekly review with accept and reject buttons.
- **As a pull request (optional):** accepting creates a branch and a PR against `routing.yaml` through `gh`, with the evidence in the PR body. No attribution trailers, per the Harness's commit rules.
- **Feedback loop:** after an applied change, PU compares the affected metric over the next 2 weeks and records whether it moved as predicted. That history becomes the training ground for PU's planned auto-skill and rule generation.

Product boundaries and privacy

The Harness and PU stay separately useful and separately sellable. The event schema is the only contract between them, and PU's local tier works fully offline.

What lives where

Capability	Harness plugin	PU local (free)	PU hub (paid)
Routing, escalation, eval, commit gate	Yes	No	No
Emit Harness Events v1	Yes	No	No
Ingest, validate, store events	No	Yes	Yes (pushed)
Cost attribution per task	No	Yes	Yes
Weekly review	No	Yes, per machine	Yes, across machines and repos
Baseline comparison	No	Yes	Yes
Calibration suggestions and PRs	No	Yes	Yes, with cross-repo evidence
Team aggregate view	No	No	Yes, opt-in

Third-party consumers

Because the stream is documented and versioned, anyone can build on it: a Grafana dashboard reading the JSONL, a CI job that fails on rising escalation rates, a competitor to PU. That's intended. PU competes on the joins nobody else has: transcripts, session summaries and handover memory linked to every task.

Privacy and governance

- **User-owned by default.** Events and reviews stay on the developer's machine unless that developer pushes them.
- **No individual tracking.** A team view shows aggregates per repository and tier, never per person. A manager can request a developer's review; the developer decides whether to share it. PU never pulls it.
- **Minimal by construction.** The stream already excludes code, prompts and diffs. PU adds cost and links to the user's own session summaries, which follow PU's existing transcript rules.

- **Deletion.** Deleting a repo from PU's registry removes its events, projections and suggestions locally, and from the hub if pushed.

Implementation plan

PU work runs one step behind the Harness: the collector must be ready before the Harness's observe-mode baseline starts (Harness Phase 1.5), and the comparison report is needed when routing goes live (Harness Phase 2). Sizes are rough estimates for one engineer.

Phase 1 — Collector and local store (2–3 days)

Should be ready before the Harness enters observe mode.

- ☐ Repo registry with automatic discovery from session working directories
- ☐ Cursor-based tailing with rotation handling
- ☐ Validation against `harness.events.v1.json` from the plugin; quarantine table
- ☐ Idempotent upsert on `event_id`; re-ingesting a whole spool creates no duplicates
- ☐ Ingestion wired into PU's existing `SessionEnd` / `Stop` hooks and the daily ETL
- ☐ Retention warning when a repo hasn't been ingested for 15 days

Phase 2 — Projections and cost attribution (2–3 days)

- ☐ `harness_tasks` and `harness_plans` rebuilt from raw events, with `complete` flags
- ☐ Subagent transcripts parsed for usage per call; versioned price table
- ☐ Agent-to-task linking through `subagent.stopped.task_ids`
- ☐ Orchestration overhead computed per plan and shown separately
- ☐ Hand-check: costs for 5 tasks match a manual calculation from transcripts within 1%

Phase 3 — Weekly review (2 days)

- ☐ `pu harness review --week` prints all ten metrics
- ☐ PU view with headline, tier table, cost split and exceptions
- ☐ Exceptions link to PU session summaries and handover memory

Phase 4 — Baseline comparison (1–2 days)

Needed when the Harness switches from observe to route.

- ☐ Stratified per-band comparison with weighted headline
- ☐ Sample-size warnings and price-change flags
- ☐ Orchestration overhead shown against routing savings

Phase 5 — Calibration suggestions (2–3 days)

- ☐ Rules R1–R6 evaluated weekly with the one-change-per-key constraint
- ☐ Suggestion records with evidence; accept and reject in the UI
- ☐ Optional PR against `routing.yaml` through `gh`, attribution-free
- ☐ Post-change tracking of whether the metric moved as predicted

Phase 6 — Hub sync and team view (later, paid tier)

- ☐ Push events and projections to Cosmos DB with the partition keys above
- ☐ Cross-machine and cross-repo weekly review
- ☐ Opt-in team aggregate view with no per-person breakdown
- ☐ Export of the event stream for third-party tools

Risks and open questions

The main risk is attribution quality: if the agent-to-task link or the transcript parsing is off, every cost figure downstream is wrong. Phase 2's hand-check exists for that reason.

Risks

Risk	Impact	Mitigation
Missing or malformed report headers break agent-to-task links	Costs land in an "unattributed" bucket	<code>require-report</code> already enforces the header; PU reports the unattributed share and flags it above 5%
Transcript format changes in a Claude Code release	Cost parsing breaks silently	Parser tests against sample transcripts per Claude Code version; alert when a session yields zero priced calls

Risk	Impact	Mitigation
Small samples drive calibration	Thresholds oscillate	Minimum 20 tasks per rule, one change per key, 7-day cool-down
Spool rotation before ingestion	Permanent data loss	Retention warning at half the window; ingestion on every session end
The review turns into surveillance	Developers stop trusting the tool	User-owned data, aggregates only in team views, request-not-pull sharing

Producer requests for Appendix A

Additive v1 changes PU needs from the Harness:

- ☒ `task_ids[]` on `subagent.stopped`, parsed from report headers (already added to Appendix A)
- ☐ `config_sha256` in the envelope: hash of the `routing.yaml` in effect, so PU can tell when an accepted suggestion is live and attribute metric changes to config versions (now added to Appendix A, on both outputs)
- ☐ `release.failed` is optional and comes from the Harness's own CI step; when it's absent, R5 uses deployed-stage failures instead

Open questions

- **Local store engine:** SQLite for simplicity, or DuckDB for faster weekly aggregations over large event volumes?
- **Multi-repo plans:** confirm how PU stitches one `plan_id` across several repos' spools, especially when repos are on different machines.
- **Price table source:** maintained by hand in PU, or fetched from a published source with manual override?
- **Subscription users:** is list-price equivalent the right primary number, or should token totals lead for them?
- **Retention in PU:** how long raw events are kept locally and on the hub, and whether projections outlive them.